



TRABAJO PRÁCTICO INTEGRADOR

Gestión de Datos de Países en Python: filtros,
ordenamientos y estadísticas

PROGRAMACION I
TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

Gonzalez Nahir Evelyn, Rossi Franco Ian
43113814 nahirnahir2025@outlook.com – 42559183 rossifrancoian@gmail.com

INDICE:

Marco Teórico.....	2
Listas en Python	2
Diccionarios en Python	3
Definición de funciones en Python	4
Condicionales en Python.....	6
Ordenamientos	7
Estadísticas básicas	8
Lectura y escritura de archivos CSV	9
Bucles for en Python	10
Ciclo while en Python.....	11
Errores y Excepciones en Python	12
Decisiones de Diseño, Arquitectura y Técnicas Utilizadas	14
Arquitectura General	14
Diagrama de flujo de operaciones principales	15
Estructura de Datos	15
Persistencia de Datos	16
Validación de Entradas.....	16
Manejo de Excepciones	16
Técnicas de Programación Utilizadas.....	17
Criterios de Calidad Aplicados.....	17
Capturas de pantalla de la ejecución de ejemplos.....	18
Dificultades Encontradas y Conclusiones.....	19
Dificultades Encontradas	19
Conclusiones	20
Anexo: (Repositorio GitHub y Video demostración)	22
Bibliografía.....	23
Listas en Python	23
Diccionarios en Python	23
Definición de funciones en Python	23
Condicionales en Python.....	23
Ordenamientos	23
Estadísticas Básicas	23
Lectura y escritura de archivos CSV	23
Bucles for en Python	23
Ciclo while en Python.....	23
Errores y Excepciones en Python	23

Marco Teórico

Listas en Python

Python ofrece distintos tipos de datos compuestos para agrupar valores, en este caso las listas se utilizan para agrupar valores separados por comas dentro de corchetes, a estos valores se los denomina comúnmente como elementos.

En el proyecto, la lista será la estructura principal que contendrá todos los países cargados en el sistema.

```
países = [] #lista vacia donde se almacenaran los países
```

Una lista puede ser inicializada como vacía mediante corchetes [] sin ningún elemento o puede definirse con elementos ya establecidos.

Pueden ser homogéneas (contienen elementos del mismo tipo de dato) o heterogéneas (pueden contener diferentes tipos de datos simultáneamente), aunque normalmente se las utiliza para almacenar datos homogéneos.

Acceso a los elementos: Las listas permiten acceder a sus elementos mediante índices (empezando en 0) y también admiten índices negativos para contar desde el final. A su vez, se pueden extraer subconjuntos de elementos mediante el slicing(cortado en rodajas), esta operación devuelve una lista nueva, es decir, no modifica la original. Esto será útil, por ejemplo, para mostrar una porción del dataset sin afectar la lista completa.

```
países = [
    {"nombre": "Argentina", "poblacion": 45376763, "superficie": 2780400, "continente": "América"},
    {"nombre": "Japón", "poblacion": 125800000, "superficie": 377975, "continente": "Asia"},
    {"nombre": "Brasil", "poblacion": 213993437, "superficie": 8515767, "continente": "América"},
]

print(países[0]) # primer país de la lista
print(países[-1]) # último país de la lista
print(países[0:2]) # los primeros dos países, sin modificar la lista original
```

Mutabilidad: Las listas son mutables, es decir es posible cambiar su contenido después de ser creadas. Su mutabilidad nos permite reemplazar valores en posiciones específicas, agregar elementos nuevos mediante método `append()`, o incluso alterar el tamaño de la lista asignando valores a un rango de índices.

```
# Agregar un nuevo país al sistema
nuevo_pais = {"nombre": "Alemania", "poblacion": 83149300, "superficie": 357022, "continente": "Europa"}
países.append(nuevo_pais)
print(países)

# Actualizar la población de un país ya existente
países[0]["poblacion"] = 46000000
print(países[0])

# Vaciar la lista completamente
países[:] = []
print(países) # []
```

Función len(): La función incorporada len() también se aplica a listas. En el sistema, esto será útil para saber cuántos países están cargados o cuántos resultados devolvió un filtro.

```
print(f"Total de países en el sistema: {len(paises)}")
```

Diccionarios en Python

El diccionario es otro tipo de dato compuesto que ofrece Python. A diferencia de las listas y otras secuencias, donde los elementos se acceden mediante un rango numérico de índices, los diccionarios se indexan mediante claves, lo que permite asociar cada clave con un valor. En el proyecto, cada país será representado como un diccionario, donde las claves serán los campos del registro: nombre, población, superficie y continente.

Las claves de un diccionario deben ser de tipo inmutable: pueden ser cadenas de texto, números, o incluso tuplas, siempre que estas últimas contengan únicamente elementos inmutables (cadenas, números u otras tuplas). En cambio, no se pueden usar listas como claves, justamente porque son mutables y las claves necesitan estabilidad.

Estructura básica: Conviene pensar un diccionario como un conjunto de pares clave:valor, donde las claves no pueden repetirse dentro del mismo diccionario. Un diccionario vacío se crea con un par de llaves {}, y para inicializarlo con datos se puede escribir una lista de pares clave:valor separados por comas, también dentro de las llaves.

```
pais = {  
    "nombre": "Argentina",  
    "poblacion": 45376763,  
    "superficie": 2780400,  
    "continente": "América"  
}  
print(pais)
```

Operaciones principales: Las operaciones básicas sobre un diccionario son almacenar un valor asociado a una clave, o extraer el valor a partir de la clave. También es posible eliminar un par clave:valor utilizando la palabra clave "del". Si se almacena un valor en una clave que ya existía, el valor anterior se pierde, ya que queda sobrescrito por el nuevo. Estas operaciones serán esenciales al momento de actualizar los datos de población y superficie de un país ya registrado en el sistema.

```
# Acceder a los datos de un país  
print(pais["nombre"])      # Argentina  
print(pais["poblacion"])   # 45376763  
  
# Actualizar población y superficie  
pais["poblacion"] = 46000000  
pais["superficie"] = 2780400  
print(pais)  
  
# Eliminar un campo  
del pais["continente"]
```

Listar y verificar la existencia de una clave: Aplicar list() sobre un diccionario devuelve sus claves en orden de inserción, y sorted() las devuelve ordenadas. El operador in permite verificar si una clave existe en el diccionario, lo cual será útil para validar que un país tenga todos sus campos antes de ser agregado al sistema.

```
#Listar y verificar existencia de claves
print(list(pais))          # ['nombre', 'poblacion', 'superficie', 'continente']
print("nombre" in pais)    # True
print("capital" in pais)   # False
```

Crear diccionarios mediante dict(): El constructor dict() permite crear diccionarios a partir de una secuencia de pares clave-valor, por ejemplo al leer cada fila del archivo CSV.

```
#Construir diccionario a partir del CSV
fila_csv = [("nombre", "Japón"), ("poblacion", 125800000), ("superficie", 377975), ("continente", "Asia")]
pais = dict(fila_csv)
print(pais) # {'nombre': 'Japón', 'poblacion': 125800000, 'superficie': 377975, 'continente': 'Asia'}
```

Comprensión de diccionarios: Python permite construir diccionarios de forma compacta usando comprensiones, útiles para transformar datos o generar resúmenes, como contar la cantidad de países por continente.

```
#Recuento de países por continente
países = [
    {"nombre": "Argentina", "continente": "América"},
    {"nombre": "Brasil", "continente": "América"},
    {"nombre": "Alemania", "continente": "Europa"},
]

cantidad_por_continente = {
    continente: sum(1 for p in países if p["continente"] == continente)
    for continente in set(p["continente"] for p in países)
}
print(cantidad_por_continente) # {'América': 2, 'Europa': 1}
```

Definición de funciones en Python

Existe una estrategia de diseño conocida como modularización, la cual consiste en dividir un problema complejo en partes más pequeñas, independientes y manejables.

Las funciones son bloques de código que implementan esta modularidad. Al definir una función estamos encapsulando una secuencia de instrucciones para realizar una tarea específica. En el proyecto, cada funcionalidad del sistema —agregar, buscar, filtrar, ordenar, mostrar estadísticas— estará encapsulada en su propia función, siguiendo el principio de que cada función tiene una única responsabilidad.

Para definir una función en Python se utiliza la palabra clave def, seguida del nombre que se le quiera dar a la función y, entre paréntesis, la lista de parámetros que recibirá. Las instrucciones que se ejecutarán son el cuerpo de la función y se escriben en las líneas siguientes. Y es obligatorio que estén indentadas con respecto a la línea del def.

```
#Ejemplo definicion de funciones
def agregar_pais(lista, nombre, poblacion, superficie, continente):
    """Agrega un nuevo país a la lista del sistema."""
    nuevo = {
        "nombre": nombre,
        "poblacion": poblacion,
        "superficie": superficie,
        "continente": continente
    }
    lista.append(nuevo)
    print(f"País '{nombre}' agregado correctamente.")
```

Docstrings: De forma opcional, pero siguiendo con lo que llamamos buenas prácticas, podemos iniciar la primera instrucción dentro del cuerpo de una función con una cadena de texto llamada docstring, la cual cumple la función de documentación.

Tabla de símbolos locales y alcance de variables: Cada vez que se ejecuta una función, Python crea una tabla de símbolos local para sus variables. Cuando se asigna un valor a una variable dentro de la función, ese valor queda guardado localmente y no afecta al resto del programa. Para modificar una variable global desde dentro de una función es necesario declararla con global.

```
#Tabla de simbolos locales
total_paises = 0
def registrar_pais(lista, pais):
    """Registra un país y actualiza el contador global."""
    global total_paises
    lista.append(pais)
    total_paises += 1

paises = []
registrar_pais(paises, {"nombre": "Chile", "poblacion": 19000000, "superficie": 756102, "continente": "América"})
print(f"Total de países registrados: {total_paises}") # 1
```

Sentencia return: La sentencia return permite que una función devuelva un resultado al código que la llamó. En el proyecto, las funciones de búsqueda y filtrado devolverán listas con los países que cumplan los criterios indicados, mientras que las funciones de estadísticas devolverán valores calculados.

```
#Ejemplo Sentencia return
def buscar_por_nombre(lista, nombre):
    """Busca países cuyo nombre contenga el texto ingresado."""
    resultados = []
    for pais in lista:
        if nombre.lower() in pais["nombre"].lower():
            resultados.append(pais)
    return resultados

encontrados = buscar_por_nombre(paises, "ar")
print(encontrados)
```

Métodos por utilizar: Dentro de las funciones del proyecto se utilizarán métodos como `append()` para agregar países, `lower()` para comparar nombres sin distinguir mayúsculas, o `strip()` para limpiar espacios en los datos leídos del CSV.

Condicionales en Python

Sin estructuras de control, un programa se ejecutaría de forma puramente secuencial, instrucción por instrucción, sin poder alterar ese orden. Los condicionales son estructuras de control que nos permiten generar que ciertos bloques de código se ejecuten únicamente cuando se cumplen determinadas condiciones, modificando así el flujo normal del programa. En el proyecto estarán presentes en casi todas las operaciones: al validar datos ingresados, al verificar si un país existe, al aplicar filtros por continente o rangos numéricos, y al mostrar mensajes de resultado.

La sentencia `if`: El condicional `if` evalúa una condición si y, solo si esta es verdadera, ejecuta el bloque de código asociado. Por ejemplo, antes de agregar un país al sistema se puede verificar que ningún campo esté vacío.

```
#Ejemplo Sentencia if
nombre = input("Nombre del país: ")
poblacion = input("Población: ")

if nombre and poblacion:
    print("Datos válidos, procediendo a registrar...")
else:
    print("Error: no se permiten campos vacíos.")
```

Un `if` se compone entonces de dos partes: la condición a evaluar y el bloque de instrucciones que se ejecutará si dicha condición se cumple. La línea del `if` debe terminar con dos puntos

“:”, y el bloque de código de instrucciones que forma parte del condicional deberá estar indentado, de lo contrario Python lo considera fuera del bloque y se ejecutará siempre, sin importar si la condición se cumplió o no.

Condiciones combinadas: Es posible combinar varias condiciones dentro de un mismo if utilizando operadores lógicos como and. Cada condición se evalúa de forma independiente y el resultado final será True solo si todas se cumplen (en el caso de and). Esto será útil al filtrar países por rango de población o superficie, donde se deben cumplir dos condiciones simultáneamente:

```
#Ejemplo condiciones combinadas
def filtrar_por_poblacion(lista, minimo, maximo):
    """Filtra países cuya población esté dentro del rango indicado."""
    resultados = []
    for pais in lista:
        if pais["poblacion"] >= minimo and pais["poblacion"] <= maximo:
            resultados.append(pais)
    return resultados
```

Clausulas else y elif: Else se utiliza cuando necesitamos ejecutar algo cuando la condición establecida no se cumple. En el caso de que necesitemos evaluar múltiples casos posibles, se utiliza elif. En el menú del sistema, esto permitirá dirigir cada opción elegida por el usuario hacia la función correspondiente.

```
#Ejemplo Sentencias else y elif
opcion = input("Seleccioná una opción: ")
if opcion == "1":
    print("Agregando país...")
elif opcion == "2":
    print("Actualizando datos...")
elif opcion == "3":
    print("Buscando país...")
elif opcion == "4":
    print("Mostrando estadísticas...")
else:
    print("Opción inválida. Intentá de nuevo.")
```

Dentro de una misma estructura condicional, solo puede haber un if y, como máximo, un único else, mientras que la cantidad de elif no tiene límite.

Ordenamientos

Python ofrece dos formas principales de ordenar datos: el método list.sort(), que modifica la lista original, y la función sorted(), que genera una nueva lista ordenada sin alterar la original. En el proyecto se utilizará sorted() para presentar el dataset ordenado según el criterio elegido por el usuario, sin perder el orden de inserción original.

Orden ascendente y descendente: sorted() acepta además un parámetro booleano llamado reverse, que permite invertir el criterio de ordenamiento (de ascendente a descendente).


```
# Ordenar por nombre (ascendente)
por_nombre = sorted(países, key=lambda p: p["nombre"])

# Ordenar por población (descendente)
por_poblacion = sorted(países, key=lambda p: p["poblacion"], reverse=True)

# Ordenar por superficie (ascendente)
por_superficie = sorted(países, key=lambda p: p["superficie"])

for pais in por_nombre:
    print(pais["nombre"])
```

El parámetro `key` indica el criterio de comparación, en este caso el valor de un campo del diccionario de cada país. El parámetro `reverse=True` invierte el orden, permitiendo obtener resultados de mayor a menor. Combinando ambos parámetros el sistema podrá cubrir todos los ordenamientos requeridos: por nombre, por población y por superficie, tanto ascendente como descendente.

Estadísticas básicas

Python incluye un módulo llamado `statistics`, pensado para realizar cálculos estadísticos básicos sobre datos numéricos. En el proyecto, este módulo junto con operaciones aritméticas simples permitirá generar los indicadores requeridos: promedio de población, promedio de superficie, país con mayor y menor población, y cantidad de países por continente.

Tipos de datos compatibles: Es importante que los datos sobre los que se aplican estas funciones sean del tipo correcto. Al leer el CSV, los valores de población y superficie llegarán como cadenas de texto y deberán convertirse a enteros antes de operar con ellos. Además, si existieran valores faltantes o mal formateados en el dataset, podrían generarse resultados incorrectos, por lo que conviene limpiar los datos antes de calcular estadísticas.

```
#Ejemplo estadísticas básicas
import statistics

poblaciones = [p["poblacion"] for p in paises]
superficies = [p["superficie"] for p in paises]

print(f"Promedio de población: {statistics.mean(poblaciones)}")
print(f"Promedio de superficie: {statistics.mean(superficies)}")
print(f"País más poblado: {max(paises, key=lambda p: p['poblacion'])['nombre']}")
print(f"País menos poblado: {min(paises, key=lambda p: p['poblacion'])['nombre']}")

# Cantidad de países por continente
continentes = {}
for pais in paises:
    c = pais["continente"]
    continentes[c] = continentes.get(c, 0) + 1
print(continentes)
```

Lectura y escritura de archivos CSV

CSV (siglas de "Comma-Separated Values", o "valores separados por comas") es uno de los formatos más utilizados para importar y exportar datos entre hojas de cálculo y bases de datos. En el proyecto, el dataset de países se almacenará y leerá desde un archivo en este formato.

Se trata de un formato que se usó de manera informal durante mucho tiempo antes de que se intentara estandarizarlo. Justamente por no haber existido siempre un estándar estricto, distintas aplicaciones pueden generar o interpretar archivos CSV con pequeñas variaciones entre sí, lo cual puede generar inconvenientes al trabajar con archivos provenientes de distintas fuentes.

Python resuelve esto mediante el módulo `csv`, que permite leer y escribir datos tabulares sin necesidad de gestionar manualmente los separadores u otros detalles del formato.

Los objetos `reader` y `writer` trabajan con los datos como secuencias (listas), mientras que `DictReader` y `DictWriter` los manejan como diccionarios, lo que resulta especialmente conveniente para este proyecto ya que cada fila del CSV se corresponde directamente con el diccionario de un país.

```

#Ejemplo lectura y escritura de archivos CSV
import csv
# Leer el dataset desde el archivo CSV
def cargar_países(ruta):
    """Carga los países desde un archivo CSV y los devuelve como lista de diccionarios."""
    países = []
    with open(ruta, newline="", encoding="utf-8") as archivo:
        lector = csv.DictReader(archivo)
        for fila in lector:
            países.append({
                "nombre": fila["nombre"],
                "poblacion": int(fila["poblacion"]),
                "superficie": int(fila["superficie"]),
                "continente": fila["continente"]
            })
    return países

# Guardar el dataset actualizado en el archivo CSV
def guardar_países(ruta, países):
    """Guarda la lista de países en el archivo CSV."""
    with open(ruta, "w", newline="", encoding="utf-8") as archivo:
        campos = ["nombre", "poblacion", "superficie", "continente"]
        escritor = csv.DictWriter(archivo, fieldnames=campos)
        escritor.writeheader()
        escritor.writerows(países)
    print("Datos guardados correctamente.")

```

En este caso de ejemplo, el código sirve para leer y guardar datos en un archivo CSV de forma organizada.

Primero, con `cargar_países`, abre el archivo usando `open` con `encoding="utf-8"` para manejar correctamente caracteres como acentos, y utiliza `csv.DictReader` para leer cada fila como un diccionario donde las claves son los nombres de las columnas. Luego recorre cada fila, convierte los valores numéricos a enteros y los guarda en una lista de diccionarios.

Después, con `guardar_países`, abre el archivo en modo escritura y usa `csv.DictWriter` para escribir nuevamente los datos, primero colocando los encabezados y luego todas las filas. Se carga así el CSV porque `DictReader` permite trabajar de forma más clara y segura accediendo a los datos por nombre de columna en lugar de por posición, lo que hace el código más fácil de entender y menos propenso a errores.

En resumen, con este módulo resulta sencillo tanto leer información tabular almacenada en archivos CSV como generar nuevos archivos en ese formato a partir de datos propios del programa, sin tener que lidiar manualmente con separadores, comillas u otros detalles de bajo nivel del formato.

Bucles for en Python

El bucle `for` es una estructura de control que permite repetir un bloque de código iterando sobre los elementos de una colección. En el proyecto será la estructura de repetición más

utilizada: para recorrer la lista de países al buscar, filtrar, mostrar resultados o calcular estadísticas.

Su sintaxis básica es la siguiente: se escribe la palabra clave `for`, seguida de una variable que tomará el valor de cada elemento en cada iteración (vuelta), la palabra `in`, y el iterable a recorrer. Al igual que en los condicionales, la línea debe terminar con dos puntos (`:`) y el bloque de código a ejecutar debe estar indentado.

```
#Ejemplo bucle for
def mostrar_paises(lista):
    """Muestra todos los países cargados en el sistema."""
    for pais in lista:
        print(f"País: {pais['nombre']} | Continente: {pais['continente']} ...
```

Ciclo while en Python

El bucle `while` es otra estructura de control repetitiva que, a diferencia del `for`, no itera sobre una secuencia predefinida, sino que repite un bloque de código mientras una condición se mantenga verdadera. Cuando esa condición pasa a ser falsa, la ejecución del bucle se detiene y el programa continúa con lo que sigue.

Bucle `while` infinito en Python: Una variante particular del `while` es el uso de `while True`, que crea intencionalmente un bucle infinito. Dado que la condición siempre es verdadera, el bucle nunca se detiene por sí solo; la única forma de salir de él es mediante la instrucción `break`, que interrumpe la ejecución del bucle en el momento en que se la encuentra. En el proyecto será la base del menú principal del sistema.

Su sintaxis es simple: se escribe la palabra clave `while`, seguida de la condición a evaluar, dos puntos (`:`), y el bloque de código indentado que se ejecutará repetidamente.

```

#Ejemplo bubble while. Menu principal
def menu_principal(paises):
    """Muestra el menú y redirige al usuario según su elección."""
    while True:
        print("\n=== SISTEMA DE GESTIÓN DE PAÍSES ===")
        print("1. Agregar país")
        print("2. Actualizar datos de un país")
        print("3. Buscar país por nombre")
        print("4. Filtrar países")
        print("5. Ordenar países")
        print("6. Ver estadísticas")
        print("7. Salir")

        opcion = input("Seleccioná una opción: ")

        if opcion == "1":
            # llamar función agregar
            pass
        elif opcion == "2":
            # llamar función actualizar
            pass
        elif opcion == "7":
            print("¡Hasta luego!")
            break
        else:
            print("Opción inválida. Intentá de nuevo.")

```

En este caso cortamos el ciclo con un break en la opción 7 (salir).

Errores y Excepciones en Python

Durante la ejecución de un programa, pueden ocurrir situaciones inesperadas que interrumpen el flujo normal del código: el usuario ingresa un texto donde se esperaba un número, un archivo no existe en la ruta indicada, o se intenta dividir por cero. En Python, estas situaciones se denominan excepciones, y el lenguaje ofrece mecanismos específicos para detectarlas y gestionarlas sin que el programa se detenga abruptamente.

Estructura Try / Except:

El bloque try permite "envolver" un fragmento de código que podría generar un error. Si durante su ejecución ocurre una excepción, Python interrumpe ese bloque y busca una cláusula except que se corresponda con el tipo de error ocurrido; si la encuentra, ejecuta el código definido en ella y continúa con el resto del programa. Si no hay ningún except que coincida, el error se propaga y puede detener la ejecución.

En el contexto del proyecto, esto será fundamental al momento de solicitar datos al usuario, como la población o la superficie de un país, donde se espera un número entero pero el usuario podría ingresar texto:

```
#Ejemplo manejo de errores con try / except
while True:
    try:
        poblacion = int(input("Ingresá la población del país: "))
        break # si llega aquí, el dato es válido
    except ValueError:
        print("Error: ingresá un número entero válido. Intentá de nuevo.")
```

Múltiples cláusulas except: Es posible definir más de una cláusula except para manejar distintos tipos de errores de forma diferenciada. Esto resulta especialmente útil al leer el archivo CSV del dataset, donde pueden ocurrir distintos tipos de problemas: que el archivo no exista, que los datos no tengan el formato esperado, o cualquier otro error imprevisto.

```
#Ejemplo multiples clausulas except
import csv
try:
    with open("países.csv", newline="") as archivo:
        lector = csv.DictReader(archivo)
        for fila in lector:
            poblacion = int(fila["poblacion"])
            superficie = int(fila["superficie"])
except FileNotFoundError:
    print("Error: no se encontró el archivo CSV.")
except ValueError:
    print("Error: algún campo numérico tiene un formato inválido.")
except Exception as err:
    print(f"Error inesperado: {err}")
    raise # se relanza para no ocultar errores desconocidos
```

Se recomienda ser lo más específico posible al nombrar las excepciones a capturar, reservando Exception como último recurso para errores imprevistos, y relanzándola cuando corresponda para no ocultar problemas que deberían atenderse.

Cláusula else: La estructura try / except admite además una cláusula else, que se ejecuta únicamente si el bloque try finalizó sin generar ninguna excepción. Su ventaja es que permite separar claramente el código "exitoso" del código de manejo de errores, evitando capturar accidentalmente excepciones que no corresponden al fragmento crítico.

```
#Ejemplo clausula else
try:
    archivo = open("países.csv", newline="")
except FileNotFoundError:
    print("Error: el archivo no existe.")
else:
    print(f"Archivo abierto correctamente.")
    archivo.close()
```

Lanzamiento de excepciones con raise: Python también permite lanzar excepciones de forma intencional mediante la instrucción raise. Esto resulta útil para validar reglas de negocio dentro del propio código: por ejemplo, al agregar un país, se podría verificar que ningún campo esté vacío y lanzar una excepción personalizada si no se cumple esa condición.

```
#Ejemplo lanzamiento de excepciones con raise
def agregar_pais(nombre, poblacion, superficie, continente):
    if not nombre or not continente:
        raise ValueError("El nombre y el continente no pueden estar vacíos.")
    if poblacion <= 0 or superficie <= 0:
        raise ValueError("La población y la superficie deben ser valores positivos.")
    # si pasa las validaciones, se agrega el país
    return {
        "nombre": nombre,
        "poblacion": poblacion,
        "superficie": superficie,
        "continente": continente
    }
try:
    pais = agregar_pais("Argentina", 45376763, 2780400, "América")
    print("País agregado correctamente.")
except ValueError as e:
    print(f"Error al agregar el país: {e}")
```

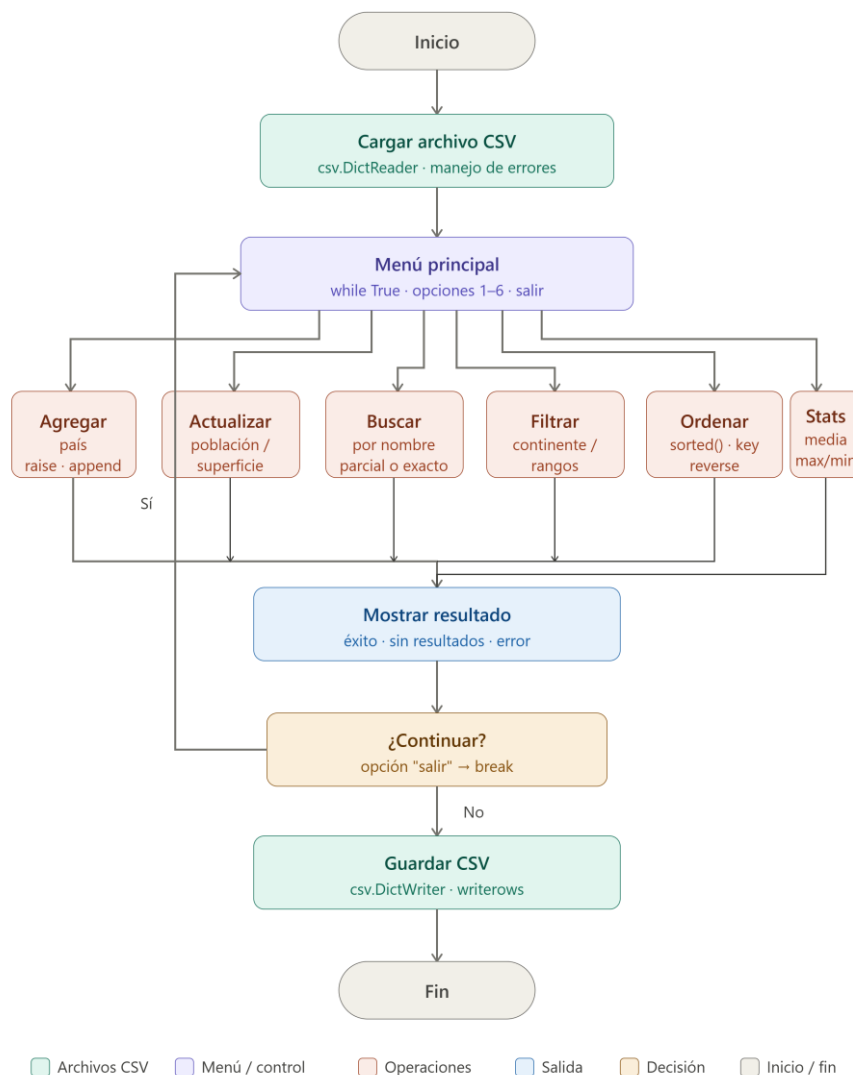
Decisiones de Diseño, Arquitectura y Técnicas Utilizadas

Arquitectura General

El sistema fue desarrollado siguiendo una arquitectura modular basada en funciones. Cada funcionalidad principal (agregar, actualizar, buscar, filtrar, ordenar y generar estadísticas) fue implementada en funciones independientes, lo que favorece la legibilidad, reutilización y mantenimiento del código.

Además, se utilizó una función principal (menu_principal) como punto de entrada del programa, encargada de coordinar la interacción con el usuario y delegar cada operación a la función correspondiente.

Diagrama de flujo de operaciones principales



- El sistema arranca cargando el archivo CSV con manejo de excepciones.
- Entra al menú principal sostenido por un `while True`.
- Desde ahí se accede a las seis operaciones: agregar, actualizar, buscar, filtrar, ordenar y estadísticas.
- Cada operación muestra su resultado (éxito, sin datos o error).
- Se pregunta si continuar: si el usuario elige salir, se rompe el ciclo con `break`, se guardan los datos en el CSV y el programa finaliza.

Estructura de Datos

Se eligió almacenar la información mediante una lista de diccionarios.

Ejemplo:

```
{
    "nombre": "Argentina",
    "poblacion": 46044703,
    "superficie": 2780400,
    "continente": "América"
}
```

Esta estructura permite:

- Acceder fácilmente a los atributos de cada país mediante claves descriptivas.
- Recorrer y procesar los registros de forma sencilla.
- Facilitar operaciones de búsqueda, filtrado y ordenamiento.
- Mantener una representación clara y cercana a los datos del archivo CSV.

Persistencia de Datos

Para la persistencia se utilizó un archivo CSV.

Esta decisión fue tomada porque:

- Es un formato simple y ampliamente utilizado.
- Puede abrirse y editarse fácilmente desde hojas de cálculo.
- No requiere instalar sistemas gestores de bases de datos.
- Resulta adecuado para datasets pequeños y medianos.

La lectura y escritura se implementó mediante el módulo estándar `csv` de Python.

Validación de Entradas

Se implementaron funciones específicas para validar los datos ingresados por el usuario:

- `pedir_entero()`
- `pedir_texto()`

Esto evita la duplicación de código y garantiza que los datos almacenados sean consistentes y válidos.

Manejo de Excepciones

Se utilizó la estructura `try/except` para controlar posibles errores durante la ejecución del programa.

Entre los casos contemplados se encuentran:

- Archivo CSV inexistente.
- Errores de conversión de datos.
- Ingresos inválidos por teclado.

- Errores inesperados de lectura o escritura.

De esta forma se evita la finalización abrupta del programa y se mejora la experiencia del usuario.

Técnicas de Programación Utilizadas

Durante el desarrollo se aplicaron diversas técnicas y herramientas del lenguaje Python:

- Funciones modulares.
- Listas y diccionarios.
- Comprensión de listas para búsquedas y filtrados.
- Funciones lambda para ordenamientos personalizados.
- Uso de módulos estándar (`csv`, `statistics` y `os`).
- Formateo de cadenas mediante f-strings.
- Validación de datos de entrada.
- Manejo de excepciones.

Criterios de Calidad Aplicados

Para mejorar la mantenibilidad y comprensión del código se adoptaron los siguientes criterios:

- Nombres descriptivos para variables y funciones.
- Separación de responsabilidades entre funciones.
- Comentarios explicativos en secciones complejas.
- Documentación mediante docstrings.
- Menús organizados jerárquicamente.
- Evitar la duplicación innecesaria de código.
- Validación sistemática de entradas del usuario.

Estas decisiones permiten que el sistema sea más fácil de entender, mantener y ampliar en futuras versiones.

Capturas de pantalla de la ejecución de ejemplos

EJEMPLO 1: AGREGAR UN PAÍS

```
=====
SISTEMA DE GESTIÓN DE PAÍSES
=====

Se cargaron 26 países desde 'países.csv'.

MENÚ PRINCIPAL
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Ver estadísticas
7. Mostrar todos los países
8. Salir

Seleccioná una opción: █
```

```
Seleccioná una opción: 1

— Agregar país —
Nombre del país   : Uruguay
Continente       : América
Población        : 3500000
Superficie (km²) : 176215

País 'Uruguay' agregado correctamente.
```

EJEMPLO 2: BUSCAR UN PAÍS

```
MENÚ PRINCIPAL
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Ver estadísticas
7. Mostrar todos los países
8. Salir

Seleccioná una opción: 3

— Buscar país —
Ingresá el nombre o parte del nombre: arg

Se encontraron 1 resultado(s):

NOMBRE                CONTINENTE      POBLACIÓN      SUPERFICIE (km²)
-----
Argentina             América        45,376,763     2,780,400
```

EJEMPLO 3: VER ESTADÍSTICAS

```
Seleccioná una opción: 6

— Estadísticas —

Total de países: 26
País más poblado: India (1,428,000,000 hab.)
País menos poblado: Nueva Zelanda (5,200,000 hab.)
Promedio de población: 197,737,568 hab.
Promedio de superficie: 3,247,078 km²

Países por continente:
América: 8
Asia: 6
Europa: 6
Oceanía: 2
África: 4

MENÚ PRINCIPAL

1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Ver estadísticas
7. Mostrar todos los países
8. Salir
```

Dificultades Encontradas y Conclusiones

Dificultades Encontradas

Durante el desarrollo del sistema surgieron distintas dificultades técnicas relacionadas tanto con la lógica del programa como con la organización del código.

Una de las primeras dificultades fue decidir cómo almacenar la información de los países. Inicialmente se evaluaron distintas alternativas, pero finalmente se optó por utilizar una lista de diccionarios porque permitía acceder a cada dato mediante claves descriptivas y facilitaba operaciones como búsquedas, filtros y actualizaciones.

Otra dificultad fue la implementación de los filtros y ordenamientos. Para resolverlos fue necesario comprender el funcionamiento de las comprensiones de listas y de las funciones lambda utilizadas por `sorted()`. En particular, entender el parámetro `key=lambda` resultó

importante para poder ordenar la información según distintos criterios sin modificar la estructura original de los datos.

También surgieron inconvenientes relacionados con la validación de entradas. El usuario podía ingresar texto donde se esperaba un número, dejar campos vacíos o introducir valores fuera de rango. Para solucionar este problema se desarrollaron funciones específicas de validación (`pedir_entero()` y `pedir_texto()`), centralizando el control de errores y evitando repetir código.

Otro aspecto desafiante fue la lectura y escritura del archivo CSV. Fue necesario convertir correctamente los datos numéricos almacenados como texto a valores enteros para poder realizar cálculos estadísticos y comparaciones. Además, se incorporó manejo de excepciones para contemplar situaciones como archivos inexistentes o errores de formato.

Finalmente, la construcción de los menús y submenús requirió un trabajo cuidadoso para mantener una navegación intuitiva y evitar salidas inesperadas del programa. El uso de ciclos `while` y estructuras condicionales permitió lograr un flujo de interacción más robusto.

Conclusiones

La realización de este proyecto permitió consolidar conocimientos fundamentales de programación en Python mediante el desarrollo de una aplicación completa y funcional.

A lo largo del trabajo se aplicaron conceptos como funciones, listas, diccionarios, manejo de archivos CSV, validación de datos, manejo de excepciones, ordenamiento de estructuras de datos y generación de estadísticas. Además, se reforzó la importancia de diseñar programas de forma modular, separando responsabilidades para obtener un código más claro y mantenible.

El proyecto también permitió comprender que gran parte del desarrollo no consiste únicamente en implementar funcionalidades, sino en prever errores, validar datos y diseñar soluciones que sean fáciles de utilizar y mantener.

Como aprendizaje general, se adquirió una mejor comprensión del proceso completo de construcción de software, desde la organización de los datos hasta la interacción con el usuario y la persistencia de la información. La experiencia resultó valiosa para afianzar conocimientos teóricos y aplicarlos en un caso práctico real.

A lo largo de este marco teórico se han explorado los conceptos fundamentales de Python que servirán como base para el desarrollo del Trabajo Práctico Integrador. Lejos de ser temas aislados, cada uno de estos elementos cumple un rol específico y complementario dentro del sistema de gestión de datos de países que se propone construir.

Las listas y los diccionarios constituyen el núcleo estructural del proyecto. Las primeras permiten almacenar colecciones ordenadas de registros, mientras que los diccionarios

posibilitan representar cada país como un conjunto de pares clave-valor, asociando de forma clara y accesible atributos como el nombre, la población, la superficie y el continente. La combinación de ambas estructuras (listas de diccionarios) nos va a permitir modelar un dataset como el que propone el trabajo.

Las funciones serán el mecanismo que permitirá organizar el código de manera modular, asignando una responsabilidad específica a cada bloque: una función para agregar países, otra para buscar, otra para filtrar, y así sucesivamente. Esta separación no solo facilita la lectura y el mantenimiento del código, sino que también reduce la probabilidad de errores al aislar cada funcionalidad.

Los condicionales estarán presentes en prácticamente todas las operaciones del sistema: al validar que los campos ingresados no estén vacíos, al verificar si un país existe antes de actualizarlo, al comparar rangos de población o superficie durante los filtros, y al mostrar mensajes de éxito o error según el resultado de cada operación.

Decidimos incorporar a este marco teórico los conceptos de ciclos for y ciclo while, ya que nos serán indispensables a la hora de recorrer el dataset de manera eficiente. El for se utilizará naturalmente para iterar sobre la lista de países al momento de buscar, filtrar o mostrar resultados. El while, por su parte, será la base del menú principal del sistema, manteniéndolo activo y a la espera de la elección del usuario hasta que este decida salir, siguiendo el patrón de while True con break estudiado en este marco.

Las técnicas de ordenamiento permitirán presentar los datos de forma significativa según el criterio que el usuario elija: alfabéticamente por nombre, o de forma ascendente o descendente según población o superficie.

Las estadísticas básicas, mediante el módulo statistics y operaciones aritméticas simples, darán lugar al módulo de indicadores del sistema: el país con mayor y menor población, el promedio de población y superficie, y la cantidad de países agrupados por continente. Estas métricas transforman los datos crudos en información útil para la toma de decisiones.

Los archivos CSV son el punto de entrada del sistema: el dataset de países se leerá desde un archivo en ese formato utilizando el módulo csv, aplicando las consideraciones vistas sobre manejo de errores de formato y limpieza de datos. Este paso es el que conecta la información externa con las estructuras internas del programa, haciendo posible que todo lo demás funcione.

Más allá de los conocimientos técnicos adquiridos, este proyecto permitió comprender que desarrollar software es un proceso de construcción progresiva en el que los errores forman parte natural del aprendizaje. Muchas de las dificultades encontradas no estaban relacionadas únicamente con la sintaxis de Python, sino con la necesidad de analizar problemas, dividirlos en partes más pequeñas y buscar soluciones de manera lógica y ordenada.

A medida que el proyecto fue creciendo, se hizo evidente la importancia de planificar antes de programar, documentar adecuadamente las funciones y mantener una estructura clara del código. Aspectos que al principio parecían simples, como validar datos o guardar información correctamente, demostraron ser fundamentales para lograr una aplicación confiable y fácil de mantener.

Además, la experiencia permitió desarrollar una mayor autonomía para investigar conceptos nuevos, interpretar mensajes de error y evaluar distintas alternativas antes de tomar una decisión de diseño. Esto ayudó a comprender que programar no consiste solamente en escribir código que funcione, sino en construir soluciones que sean comprensibles, reutilizables y robustas.

Como grupo, este trabajo dejó la enseñanza de que el aprendizaje más significativo ocurre cuando se enfrentan problemas reales y se busca resolverlos de manera colaborativa. El resultado final no solo es un programa funcional, sino también una mejor comprensión de cómo abordar proyectos de software de forma organizada y profesional.

Por último, también incluimos en el marco teórico los conceptos de manejo de errores y excepciones, ya que este será el mecanismo que le añadirá al sistema de solidez y confiabilidad. A través de bloques try / except, el programa podrá anticiparse a situaciones inesperadas sin interrumpir su ejecución: errores de formato al leer el CSV, valores no numéricos ingresados por el usuario, archivos inexistentes o campos vacíos al registrar un nuevo país. La instrucción raise, por su parte, permitirá lanzar excepciones de forma intencional dentro de las funciones de validación, estableciendo reglas claras sobre qué datos son aceptables y comunicando los problemas de manera precisa. La cláusula else, complementaria al bloque try / except, permitirá separar con claridad el flujo exitoso del flujo de error, haciendo el código más legible y menos propenso a capturar accidentalmente excepciones que no corresponden. En conjunto, estas herramientas garantizan que el sistema responda siempre con mensajes claros tanto ante operaciones exitosas como ante entradas o situaciones inválidas.

En definitiva, el Trabajo Práctico Integrador no representa simplemente la aplicación de estos conceptos de forma individual, sino su integración coordinada en una solución real: un sistema funcional, modularizado y robusto.

Anexo: (Repositorio GitHub y Video demostración)

Repositorio GitHub: <https://github.com/rossifrancoian/TPI-PROGRAMACION-ROSSI-GONZALEZ>

Video demostrativo en Youtube: <https://youtu.be/VdNxVuTuEAM>

Bibliografía

Listas en Python

<https://docs.python.org/3/tutorial/introduction.html#lists>

Diccionarios en Python

<https://docs.python.org/3/tutorial/datastructures.html#dictionaries>

Definición de funciones en Python

<https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

Condicionales en Python

<https://ellibrodepython.com/if-python>

Ordenamientos

<https://docs.python.org/es/3/howto/sorting.html>

Estadísticas Básicas

<https://docs.python.org/es/3/library/statistics.html>

Lectura y escritura de archivos CSV

<https://docs.python.org/es/3.9/library/csv.html>

Bucles for en Python

<https://elpythonista.com/bucles-en-python-for-y-while-guia-completa-2025-ejemplos>

Ciclo while en Python

<https://elpythonista.com/bucles-en-python-for-y-while-guia-completa-2025-ejemplos>

Errores y Excepciones en Python

<https://docs.python.org/es/3/tutorial/errors.html>